

Laboratorijas darbs: Apzināti sabojājam JPEG

Šis ir datorlaboratorijas darbs lekcijai 5. **Zudumradošā saspiešana: attēli un audio**. Katrs JPEG solis ir vajadzīgs kāda iemesla dēļ. Šajā darbā jūs pa vienam mainīsiet vai izņemsiet soļus, aplūkosiet atkodēto attēlu un paskaidrosiet, kas nogāja greizi.

Faili

- `jpeg_lab.py`: JPEG līdzīgs kodētājs un atkodētājs ar komandrindas parametriem katram solim (Python 3, vajag `numpy` un `Pillow`).
- Testa attēli (labais klikšķis un “Saglabāt saiti kā...”):
 - `bumblebee.png`: fotogrāfija, 400×300 pikseļi.
 - `kuldiga1.png`: fotogrāfija, 600×450 pikseļi.
 - `mandelbrot-picture.png`: datorā ģenerēts attēls ar piesātinātām krāsām un asām malām, 850×850 pikseļi.
 - `test-chart.png`: sintētiska testa tabula ar krāsu joslām, krāsainu tekstu, plānām līnijām un šaha galdiņu, 512×384 pikseļi. To uzzīmēja programma `make_test_chart.py`.
- Var izmantot arī savus attēlus. Nelielus attēlus (līdz apmēram 1000×1000 pikseļiem) ir vieglāk salīdzināt blakus, un tos apstrādā apmēram sekundē.

```
pip install numpy pillow
python jpeg_lab.py bumblebee.png out.png
python jpeg_lab.py --help
```

PNG izmanto tikai RGB pikseļu glabāšanai. `jpeg_lab.py` neveido `.jpg` failu. Tā izpilda JPEG soļus ar pikseļiem, uzreiz atkodē rezultātu un saglabā atkodētos pikseļus kā PNG attēlu. Programma arī izdrukā:

- katra krāsu kanāla kļūdu: vidējo kvadrātisko kļūdu (MSE) un $PSNR = 10 \log_{10} \frac{255^2}{MSE}$ decibelos (jo vairāk, jo labāk; ja vairāk par apmēram 40 dB, kļūdu grūti saskatīt);
- nenulles kvantizēto koeficientu skaitu;
- novērtēto saspiešamā faila izmēru. Koeficientus pārvērš *baseline* JPEG simbolos (DC starpības, pāri (RUN, SIZE), ZRL, EOB), un simbolu entropijai pieskaita papildu bitu skaitu. Ar noklusētajiem iestatījumiem novērtējums atšķiras ne vairāk kā par apmēram 5% no tādas pašas kvalitātes īsta JPEG faila izmēra, kas izmanto optimizētas Hafmana tabulas (galvenes neskaitot). Ar standarta pielikuma K Hafmana tabulām īsti faili ir lielāki, it īpaši sintētiskiem attēliem, piemēram, `test-chart.png`.

Divas papildu izvades palīdz saprast, kas notika:

- `--difference-png diff.png` ieraksta kļūdu attēlu: pelēks (128) nozīmē, ka kļūdas nav, bet gaišāki un tumšāki pikseļi – ka atkodētā vērtība ir par lielu vai par mazu (kļūdu pareizina ar `--difference-amplification`, noklusēti 4).
- `--planes-png planes.png` ieraksta trīs krāsu plaknes (piemēram, Y, Cb, Cr) kā pelēktoņu attēlus: sākotnējās plaknes augšējā rindā un atkodētās plaknes apakšējā rindā.

Vienmēr salīdziniet izmainītu palaidienu ar noklusēto palaidienu (`python jpeg_lab.py image.png default.png`) un, ja iespējams, pie vienāda novērtētā izmēra. Labāku attēlu ir viegli iegūt, iztērējot vairāk bitu.

Kā programma ir veidota

Pirmkods seko lekcijas JPEG soļiem. Lai mainītu kādu soli, atrodiet tā sadaļu failā; katrs tālāk minētais parametrs tiek glabāts `settings.<name>` (piemēram, `--block-size` ir `settings.block_size`).

Solis	Funkcijas	Parametri
Pikseļu nolasīšana	<code>load_rgb_pixels</code>	<code>input_png</code>
1. Krāsu telpas maiņa	<code>rgb_to_planes</code> , <code>planes_to_rgb</code>	<code>--color-space</code> { <code>ycbcr</code> , <code>yiq</code> , <code>rgb</code> }
2. Krāsainības izretināšana	<code>subsample</code> , <code>upsample</code>	<code>--chroma-subsampling</code> { <code>4:4:4</code> , <code>4:2:2</code> , <code>4:2:0</code> , <code>4:1:1</code> }
3. Bloki un līmeņa nobīde	<code>pad_to_multiple</code> , <code>split_into_blocks</code> , <code>merge_blocks</code>	<code>--block-size</code> , <code>--no-level-shift</code>
4. Transformācija	<code>dct_matrix</code> , <code>dst7_matrix</code> , <code>hartley_matrix</code> , <code>walsh_hadamard_matrix</code> , <code>haar_matrix</code> , <code>klt_matrix</code>	<code>--transform</code>
4b. Koeficientu atlase (JPEG tās nav)	<code>drop_high_frequencies</code> , <code>scale_up_kept_coefficients</code> , <code>fill_dropped_with_noise</code>	<code>--keep-coefficients</code> , <code>--energy-compensation</code> { <code>none</code> , <code>scale</code> , <code>noise</code> }
5. Kvantizācija	<code>make_quantization_tables</code> , <code>scale_table_by_quality</code> , <code>quantize</code> , <code>dequantize</code>	<code>--quality</code> , <code>--quant-table</code> , <code>--flat-step</code> , <code>--luma-quant-multiplier</code> , <code>--chroma-quant-multiplier</code> , <code>--rounding</code>
6.-7. Zig-zag, sēriju kodēšana, izmērs	<code>zigzag_order</code> , <code>estimate_plane_bits</code>	nav
Atkodētājs: bloku robežu gludināšana (JPEG tās nav)	<code>deblock</code>	<code>--deblocking-filter</code> , <code>--deblocking-threshold</code>

Dažas detaļas:

- Katra transformācija ir ortonormēta $N \times N$ matrica T . Bloku A pārveido par $B = TAT^T$ un atjauno kā $A = T^TBT$. DCT gadījumā T ir lekcijas matrica C .
- Ja bloka izmērs nav 8, tad 8×8 kvantizācijas tabulas izstiepj: $N \times N$ bloka koeficients (u, v) saņem tabulas elementu $Q_{\lfloor 8u/N \rfloor, \lfloor 8v/N \rfloor}$.
- `--quality` mērogo tabulas tāpat kā *libjpeg* (sk. lekcijas 5. soli). Pēc tam `--luma-quant-multiplier` pareizina pirmās plaknes tabulu, bet `--chroma-quant-multiplier` pareizina otrās un trešās plaknes tabulu.
- Atkodētājs palielina krāsainības izšķirtspēju, atkārtotot paraugus. Īsti atkodētāji bieži interpolē, tāpēc to attēli izskatās nedaudz gludāki.

1. uzdevums: Bez krāsu telpas maiņas

Ar `--color-space rgb` programma izlaiž 1. soli: R kodē tā, it kā tas būtu Y , G – kā C_b , bet B – kā C_r . Tātad G un B tiek izretināti un kvantizēti ar krāsainības tabulu, kas ir rupjāka.

```
python jpeg_lab.py bumblebee.png ycbcr.png
python jpeg_lab.py bumblebee.png rgb.png --color-space rgb
python jpeg_lab.py test-chart.png chart-ycbcr.png --planes-png
↳ chart-ycbcr-planes.png
python jpeg_lab.py test-chart.png chart-rgb.png --color-space rgb
```

1. Salīdziniet attēlus, PSNR un novērtēto izmēru. Kurš attēls ir vairāk izplūdis, un kāpēc? Atcerieties, kuram no R , G , B ir lielākais svars formulā $Y = 0.299R + 0.587G + 0.114B$.
2. Aplūkojiet `test-chart.png` vertikālās melnās līnijas abos rezultātos. No kurienes rodas krāsa, kas parādās failā `chart-rgb.png`?
3. Tagad izslēdziet izretināšanu un izmantojiet vienu un to pašu tabulu visām trim plaknēm, lai atšķirtos tikai krāsu telpa:

```
python jpeg_lab.py test-chart.png a.png --chroma-subsampling 4:4:4
↳ --quant-table luma-for-all
python jpeg_lab.py test-chart.png b.png --chroma-subsampling 4:4:4
↳ --quant-table luma-for-all --color-space rgb
```

Kurš ir mazāks? Aplūkojiet abu palaidienu `--planes-png`. Kāpēc tipiskas fotogrāfijas C_b un C_r plaknes saspiežas tik labi, bet G un B plaknes – nē?

4. Kvantizējiet otro un trešo plakni agresīvāk. Divkāršojiet krāsainības tabulu (`--chroma-quant-multiplier 2`), tad izmēģiniet 4 un 8. Dariet to abās krāsu telpās (ar `--chroma-subsampling 4:4:4`):

```
python jpeg_lab.py bumblebee.png c.png --chroma-subsampling 4:4:4
↳ --chroma-quant-multiplier 4
python jpeg_lab.py bumblebee.png d.png --chroma-subsampling 4:4:4
↳ --chroma-quant-multiplier 4 --color-space rgb
```

Kurā krāsu telpā krāsainības plaknes var kvantizēt rupji bez redzamiem bojājumiem? Kāda veida kļūda rodas RGB telpā: nepareizs gaišums vai nepareiza krāsa?

5. Atkārtojiet 1. punktu ar `--color-space yiq`. Vai starp YIQ un $YCbCr$ ir jūtama atšķirība? Kāpēc (sk. lekciju: kā saistītas IQ un UV plaknes)?

2. uzdevums: Lielu bloku un bloku artefakti

```
python jpeg_lab.py bumblebee.png b8.png --quality 10
python jpeg_lab.py bumblebee.png b32.png --quality 10 --block-size 32
python jpeg_lab.py bumblebee.png b32-deblocked.png --quality 10 --block-size 32
↳ --deblocking-filter
```

1. Zemā kvalitātē katrā blokā paliek tikai daži koeficienti. Tas pats attiecas uz vidējo gaišumu un lēzenu slīpumu bloka iekšpusē. Kāpēc tie nesakrīt kaimiņu bloku robežās? Kāpēc režģis ir daudz labāk redzams ar 32×32 blokiem nekā ar 8×8 blokiem? Izmēģiniet arī `--quality 5`.
2. Atrodiet asu malu (lapu vai zieda ziedlapiņu) un aplūkojiet viļņošanās (*ringing*) blakus tai. Cik tālu no malas viļņošanās izplatās ar 8×8 blokiem un ar 32×32 blokiem? Ko tas pasaka par bloka izmēra izvēli (salīdziniet ar AVIF adaptīvajiem bloku izmēriem lekcijā)?

3. Bloku robežu gludināšanas filtrs (*deblocking*, funkcija `deblock`) aplūko trīs pikselus katrā bloka robežas pusē. Ja lēcians pāri robežai ir mazāks par `--deblocking-threshold` un abas puses ir gludas, lēcianu aizstāj ar lineāru pāreju. Izmēģiniet sliekšņus 10, 24, 60, 200. Kas notiek ar attēla īstajām malām, ja sliekšnis ir par lielu? Kāpēc filtrs nevar pilnībā noņemt režģi?
4. Transformācijas dēļ kļūdas var koncentrēties arī *blakus* bloku robežām. Salīdziniet kļūdu attēlus:

```
python jpeg_lab.py bumblebee.png dct16.png      --block-size 16 --difference-png
↪ dct16-diff.png
python jpeg_lab.py bumblebee.png hartley16.png --block-size 16 --transform
↪ hartley --difference-png hartley16-diff.png
python jpeg_lab.py bumblebee.png dst16.png      --block-size 16 --transform dst7
↪ --difference-png dst16-diff.png
```

Hārtlija transformācija (diskrētās Furjē transformācijas reālā versija) bloku uzskata par periodiska signāla vienu periodu, tāpēc bloka kreisā mala “turpinās” tā labajā malā. DCT uzvedas tā, it kā bloks būtu turpināts ar savu spoguļattēlu. DST-VII pieņem, ka signāls tieši pirms augšējās/kreisās malas ir 0. Katrai transformācijai uzzīmējiet bloka vienu rindu (1D) kopā ar pieņemto turpinājumu un paskaidrojiet, kur turpinājumam ir lēcians. Kur kļūdu attēlos ir lielākās kļūdas?

3. uzdevums: Paturam tikai zemās frekvences

`--keep-coefficients K` katrā blokā patur tikai augšējo kreiso $K \times K$ stūri (DC koeficientu un zemākās frekvences), bet pārējos koeficientus aizstāj ar 0. Izmantojiet augstu kvalitāti, lai kļūdu radītu koeficientu atmešana, nevis kvantizācija.

```
python jpeg_lab.py bumblebee.png keep3.png --keep-coefficients 3 --quality 90
python jpeg_lab.py test-chart.png chart-keep3.png --keep-coefficients 3 --quality
↪ 90
```

1. No 64 koeficientiem paliek 9. Kāda ir rezultāta izšķirtspēja, mērot “detaļās uz bloku”? Kuras testa tabulas daļas tiek pilnībā iznīcinātas, un kuras saglabājas? Izmēģiniet $K = 1, 2, 4, 6$.
2. Pēc Parsevāla vienādības (sk. lekciju) bloka kvadrātiskā kļūda ir tieši atmesto koeficientu kvadrātu summa. Ja $K = 3$, salīdziniet PSNR ar noklusēto palaidienu pie tā paša novērtētā izmēra (pielāgojiet noklusētā palaidiena `--quality`). Kurš attēls izskatās labāk? Ko tas pasaka par koeficientu atmešanu pēc pozīcijas, salīdzinot ar to kvantizēšanu pēc lieluma?
3. Tagad kompensējiet zaudēto enerģiju. Ar `--energy-compensation scale` kodētājs pareizina katra bloka paturētos AC koeficientus ar tādu reizinātāju, lai bloka enerģija (kvadrātu summa) nemainītos. Ar `--energy-compensation noise` atkodētājs aizpilda atmestās pozīcijas ar gadījuma skaitļiem, kuru kopējā enerģija ir tāda pati (“trokšņa aizpildīšana”, *noise filling*; kodētājs katram blokam nosūta vienu skaitli – atmesto enerģiju).

```
python jpeg_lab.py bumblebee.png keep3-scale.png --keep-coefficients 3
↪ --quality 90 --energy-compensation scale
python jpeg_lab.py bumblebee.png keep3-noise.png --keep-coefficients 3
↪ --quality 90 --energy-compensation noise
python jpeg_lab.py test-chart.png chart-keep3-noise.png --keep-coefficients 3
↪ --quality 90 --energy-compensation noise
```

Abiem attēliem ir pareiza enerģija, bet PSNR kļūst *slīkākā*. Kāpēc? Vai kāds no attēliem izskatās asāks? Kur troksnis palīdz (zāle, lapas) un kur tas kaitē (vienmērīgi apgabali, līnijas, teksts)?

4. Audio kodeki (AAC, Opus) augstajām frekvencēm izmanto trokšņa aizpildīšanu, bet attēlu kodeki malām to gandrīz nekad neizmanto. AV1 un AVIF no jauna sintezē tikai fotogrāfijas graudainību (*film grain*). Paskaidrojiet, kāpēc šī ideja labāk der šņākoņai un graudainībai nekā līnijām un tekstam.

4. uzdevums: Kāpēc tieši šādas detaļas? Noapaļošana, tabula un līmeņa nobīde

Katrs punkts maina vienu nelielu 3.-5. soļa detaļu.

(a) Noapaļošana. JPEG noapaļo $B_{u,v}/Q_{u,v}$ līdz *tuvākajam* veselajam skaitlim.

```
python jpeg_lab.py bumblebee.png floor.png --rounding floor
python jpeg_lab.py bumblebee.png trunc.png --rounding toward-zero
```

1. Ar `floor` attēls tiek sabojāts un izmērs palielinās, lai gan mainīta tikai noapaļošana. Paskaidrojiet, kas notiek ar nelielu negatīvu koeficientu, piemēram, $B/Q = -0.1$. Cik ir nenulles koeficientu, salīdzinot ar noklusēto palaidienu?
2. Ar `toward-zero` (atmetot daļu aiz komata) attēls ir tikai nedaudz sliktāks, bet izmērs ir mazāks. Salīdziniet to ar noklusēto palaidienu pie *tāda paša* izmēra (zemāka noklusētā palaidiena `--quality`). Īsti kodētāji dažkārt apzināti noapaļo mazas vērtības uz nulles pusi (*dead zone*) vai izvēlas noapaļošanas virzienu katram koeficientam atsevišķi (*trellis quantization* AVIF formātā). Kāpēc tas var būt labs kompromiss?

(b) Kvantizācijas tabulas forma. Standarta tabulā zemajām frekvencēm ir mazi soļi, bet augstajām frekvencēm – lieli.

```
python jpeg_lab.py bumblebee.png flat.png --quant-table flat --flat-step 30
```

Atrodiet tādas `--flat-step` un `--quality` vērtības, lai palaidienam ar vienmērīgo tabulu un palaidienam ar standarta tabulu būtu vienāds novērtētais izmērs. Salīdziniet PSNR un salīdziniet attēlus. Vai augstāks PSNR arī izskatās labāk? Kāda veida kļūdu acs pamana vispirms: troksni smalkā faktūrā vai kļūdas lielos vienmērīgos apgabalos?

(c) Līmeņa nobīde. JPEG pirms DCT no katra parauga atņem 128.

```
python jpeg_lab.py bumblebee.png no-shift.png --no-level-shift
```

Rezultāts ir gandrīz identisks. Parādiet, ka ortonormētai transformācijai, kuras pirmā bāzes funkcija ir konstante, 128 atņemšana maina tikai DC koeficientu (par cik, ja $N = 8$?). Kāpēc standarts to tomēr dara? Norāde: $B_{0,0}$ vērtību diapazons un tas, ka DC vērtības kodē kā starpības.

(d) Bez transformācijas. `--transform identity` kvantizē pašus pikselus (“koeficienti” ir pikseļu vērtības).

```
python jpeg_lab.py bumblebee.png identity.png --transform identity
python jpeg_lab.py bumblebee.png identity-flat.png --transform identity
↪ --quant-table flat --flat-step 40
```

Paskaidrojiet rakstu pirmajā attēlā (kuru tabulas elementu izmanto kuram pikselim?). Otrajā attēlā katru pikseli vienkārši noapaļo līdz 40 daudzkārtņim. Kāpēc tas ir lielāks *un* sliktāks par noklusēto palaidienu? Ko dara transformācija, ko nevar izdarīt, kvantizējot pikselus?

5. uzdevums: Citas ortogonālas transformācijas

Aizstājiet DCT ar citām ortonormētām bāzēm. Visas tās izmanto praksē:

--transform	Bāzes funkcijas	Kur izmanto
dct	kosinusi (DCT-II)	JPEG, MPEG-2, H.264/H.265 (veselo skaitļu tuvinājumi), AV1
dst7	sinusi, 0 pie augšējās/kreisās malas (DST-VII)	H.265 4 × 4 intra gaišuma bloki, AV1 (ADST)
hartley	cos + sin (reāla DFT)	spektrālā analīze; tuvs FFT radinieks
walsh-hadamard	tikai +1 un -1 (sakārtotas pēc zīmes maiņu skaita)	H.264 (DC koeficienti), video kodētāji izmaksu novērtēšanai (SATD), CDMA kodi
haar	īsi pakāpieni dažādos mērogos (vienkāršākais vilnītis)	attēlu kodēšana ar vilnišiem (JPEG 2000 visam attēlam izmanto gludākus vilnišus)
klt	šr attēla pikseļu korelācijas matricas īpašvektori (PCA)	teorētiskais optimums; seju atpazīšana ("eigenfaces"), datu analīze
identity	atsevišķi pikseli	bez transformācijas (sk. 4d)

```
python jpeg_lab.py bumblebee.png t.png --transform walsh-hadamard --print-tables
↪ --difference-png t-diff.png
```

1. Katrai transformācijai pierakstiet PSNR, nenulles koeficientu skaitu un izmēru. Ierakstiet tos tabulā un sakārtojiet pēc izmēra. Izmantojiet vismaz divus attēlus (fotogrāfiju un `test-chart.png`).
2. Aplūkojiet `walsh-hadamard` un `haar` attēlus. Kāda ir tipiskā artefaktu forma, un kā tā izriet no bāzes funkciju formas (izmantojiet `--print-tables`, lai redzētu matricas rindas)?
3. `klt` aprēķina labāko ortonormēto transformāciju dotā attēla rindām un kolonnām. Salīdziniet tās izdrukāto matricu (`--block-size 4 --transform klt --print-tables`) ar lekcijas DCT matricu, ja $N = 4$. Ko jūs pamanāt? Kāpēc tas izskaidro, ka JPEG izmanto fiksētu DCT un nesūta katram attēlam savu KLT matricu?
4. Kvantizācijas tabula ir veidota DCT koeficientiem. Vai salīdzinājums ir godīgs pret citām transformācijām? Atkārtojiet salīdzinājumu ar `--quant-table flat` pie vienāda izmēra.
5. (*Neobligāti, programmēšana.*) Pievienojiet funkcijai `transform_matrix` savu transformāciju (piemēram, DCT-IV vai līdz veseliem skaitļiem noapaļotu DCT kā H.264, pēc tam normētu). Pārbaudiet, ka $T @ T.T$ ir vienības matrica.

6. uzdevums (neobligāts): Jūsu pašu modifikācija

Izmainiet kodu vienā vietā un pirms palaišanas paredziet rezultātu. Dažas idejas:

- Pārraidiet *zig-zag* secību apgrieztā virzienā vai izmantojiet secību pa rindām. `estimate_plane_bits` izmanto `zigzag_order`. Vai novērtētais izmērs mainās? Kāpēc?
- Funkcijā `quantize` pirms noapaļošanas pieskaitiet gadījuma troksni (*dithering*). Vai tas palīdz pret 2. uzdevuma režģi?
- Palieliniet krāsainības izšķirtspēju ar lineāru interpolāciju, nevis atkārtotānu (funkcija `upsample`).
- Rupji kvantizējiet tikai katru otro bloku. Vai redzat šaha galdiņu?

Katru eksperimentu apkopojiet divos vai trijos teikumos: ko jūs mainījāt, ko gaidījāt, ko redzējāt un kurš īstā JPEG solis novērš šo problēmu.